EEE 598: Generative AI: Theory and Practice

Final Project — Denoising Diffusion Implicit Models: Accelerated Sampling and Quality Trade-offs

Anish Verma (Lead) — ASU ID 1220864892
Jui-Chi Liu — ASU ID 1238358162
Aaldin Kesavan Helen — ASU ID 1237312286

First rendition: the SOL DDIM batch job is still running, so final FID numbers are not inserted yet.

Project Topic, Paper reference(s)

- Broad category: diffusion models / generative methods, with a practical focus on faster sampling.
- Main paper: Song, J., Meng, C., & Ermon, S. (2021). Denoising Diffusion Implicit Models. ICLR 2021. arXiv:2010.02502.
- What we benchmarked: DDIM vs DDPM sampling on CIFAR-10 across step counts and eta settings, using the official implementation path plus our benchmark wrapper.
- Related references we actually use in this project: Ho et al. (2020) for DDPM, Heusel et al. (2017) for FID, and Jennewein et al. (2023) for Sol.

We kept the scope centered on reproduction + benchmarking, not inventing a new diffusion model.

Key Ideas from the Paper (Theory)

- DDPM starts from a Gaussian forward diffusion chain that gradually corrupts a clean image x0 into xt.
- Because xt has a closed-form marginal, the model can be trained as a noise predictor epsilon_theta(xt, t).
- The important DDIM idea is that we can change the sampling family without changing that training objective.

Same trained network. Different sampling process.

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t}\, \epsilon$$

$$\mathcal{L} = \sum_{t=1}^{T} \lambda_t \, \mathbb{E}_{x_0, \epsilon} \left[\| \epsilon - \epsilon_\theta(x_t, t) \|^2 \right]$$

For us, this matters because the paper claims we can get a much faster sampler while keeping the same model training recipe underneath.

Key Ideas from the Paper (Theory)

- At sampling time, the network predicts a denoised estimate x-hat_0 from the noisy xt.
- DDIM then updates xt-1 with a non-Markovian step whose noise level is controlled by eta.
- eta = 0 gives deterministic DDIM sampling, while eta = 1 recovers the DDPM variance.
- This eta knob is exactly what we sweep in our benchmark because it controls the speed/quality trade-off.

$$\hat{x}_0 = \frac{x_t - \sqrt{1-\bar{\alpha}_t}\,\epsilon_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}}$$

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}}\hat{x}_0 + \sqrt{1-\bar{\alpha}_{t-1}-\sigma_t^2}\,\epsilon_\theta(x_t, t) + \sigma_t z$$

$$\sigma_t^2 = \eta^2\,\frac{(1-\alpha_t)(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}$$

The paper's practical claim is simple: fewer steps can still work if the sampling path is chosen carefully.

Goal of Project

- Reproduce the core DDIM comparison instead of proposing a new model: can we sample much faster than standard DDPM and still keep usable image quality?
- Keep the project mostly practical, but explain enough of the theory to justify why DDIM can reuse the same training objective.
- Benchmark FID and inference time for steps {10, 25, 50, 100, 250, 1000} with eta = 0 and eta = 1.
- Run a separate eta sweep at 50 steps for eta in {0.0, 0.25, 0.5, 0.75, 1.0}.
- Be honest about status: this deck is the first rendition, and the additional SOL batch job is still in progress.

Your Dataset(s)

Dataset used in this benchmark: CIFAR-10

- We use CIFAR-10 together with a pretrained DDPM checkpoint in the official DDIM setup.
- What we evaluate on this dataset: generated samples, FID, and inference time across different step counts and eta values.
- Why it fits this project: it is the exact benchmark named in our proposal and keeps the DDIM vs DDPM comparison manageable on SOL.
- Outputs we save from the runs: benchmark_results.json and auto-generated figures under figures/.

Model source

Pretrained CIFAR-10 DDPM checkpoint

Metrics

FID and inference time

Saved artifacts

benchmark_results.json
figures/
generated sample grids

We did not add outside dataset images here because we only wanted to use the material we actually submitted.

ASU SOL Compute Setup

- We cloned our benchmark repo and the official ermongroup/ddim repo, then uploaded the combined workspace to SOL.
- Because the login node has memory limits, we submitted environment setup as a batch job instead of doing everything interactively.
- Account: class_eee59838068spring2026.
- Compute target used in this project: A100 GPU, public partition, public QOS.

```
git clone https://github.com/Rich627/ddim-benchmark-cifar10.git
cd ddim-benchmark-cifar10
git clone https://github.com/ermongroup/ddim.git
scp -r ./* <asurite>@solasu.edu:"~/ddim-benchmark/"
```

```
ssh <asurite>@solasu.edu
cd ~/ddim-benchmark
sbatch setup_env.sbatch
squeue -u $USER
cat setup_env_*.out
```

This setup detail matters because the project is as much about practical benchmarking on real compute as it is about the paper itself.

Training (Model Learning) and Sampling Pipeline

Reproduction pipeline

Clone repos
→ upload to SOL
→ build env with sbatch
→ launch benchmark
→ save JSON + figures

Per-sample DDIM / DDPM path

Sample x_T ~ N(0, I)
→ predict epsilon-hat = epsilon_theta(x_t, t)
→ compute x-hat_0
→ update x_t-1 using eta
→ repeat until x_0

$$\hat{x}_0 = \frac{x_t - \sqrt{1-\bar{\alpha}_t}\,\hat{\epsilon}}{\sqrt{\bar{\alpha}_t}}$$

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}}\hat{x}_0 + c_1 z + c_2 \hat{\epsilon}$$

In our project we are benchmarking sampling, not retraining the network from scratch. The trained checkpoint stays fixed, and the main knobs we change are the step count and eta.

That is why this paper is useful: it lets us study a new sampler while keeping the learned model underneath unchanged.

Detailed Problem Setup

- Model under test: a pretrained CIFAR-10 DDPM checkpoint; our comparison is entirely about inference, not re-training.
- Libraries / code path: our benchmark.py wrapper, the official ermongroup/ddim implementation, and SLURM scripts on SOL.
- Metrics: FID and inference time, with generated plots written automatically to figures/.
- Practical constraint: the login node memory limits pushed us toward a batch-based workflow even for env setup.

```
python -u benchmark.py
python -u benchmark.py --resume
python -u benchmark.py --fid-only
python -u benchmark.py --plot-only
```

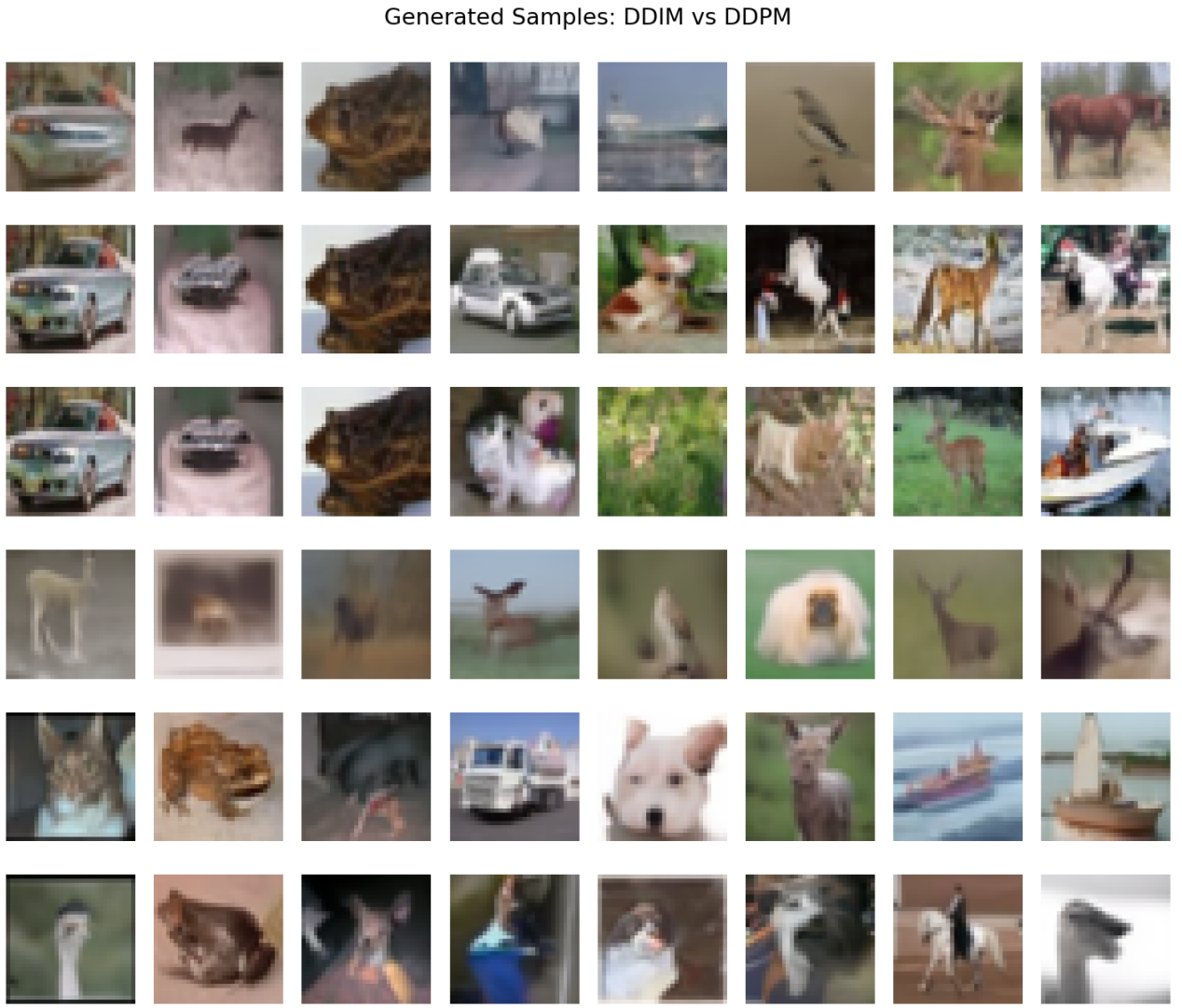
```
sbatch run_benchmark.sbatch
squeue -u $USER
tail -f benchmark_*.out
scp
<asurite>@solasu.edu:"~/ddim-benchmark/benchmark_results.j
son" .
scp -r <asurite>@solasu.edu:"~/ddim-benchmark/figures" .
```

The benchmark script was written so we can resume interrupted runs, recompute FID only, or regenerate plots without rerunning everything.

Main Results

Preliminary outputs from our first completed benchmark render

- The timing curve already shows the core runtime story: step count is the dominant cost driver.
- At the same step count, DDIM (eta = 0) and DDPM (eta = 1) look almost identical in runtime in our current plot.
- Our generated sample grid is a solid sanity check that the pipeline is producing recognizable CIFAR-10-like outputs, even though fidelity is still mixed.
- We are not inserting final FID claims yet because the additional SOL batch job is still running.



Hyperparameters and Ranges

Sampling steps

10, 25, 50, 100, 250, 1000
Main comparison axis for speed vs quality.

Beta schedule

Default linear schedule: beta_1 = 1e-4, beta_1000 = 0.02.
We kept this fixed to match the stated DDIM/DDPM setup in our implementation notes.

Eta

0.0 and 1.0 for DDIM vs DDPM; plus 0.25, 0.5, 0.75 at 50 steps for the eta sweep.
This is the stochasticity knob we explicitly tune.

Dataset / checkpoint

CIFAR-10 + pretrained DDPM checkpoint.
This stays fixed while we only change the sampler settings.

Samples per benchmark

50K samples. This stays fixed so the runtime / FID comparison is consistent.

Metrics / outputs

FID, inference time, generated sample grids, benchmark_results.json, and auto-generated figures/.

These are the exact knobs we vary in the submitted benchmark; we did not invent extra settings beyond what is in our proposal / math / code notes.

Discussion of Results

- What worked: we got an end-to-end SOL workflow that can set up the environment, submit runs, resume interrupted jobs, and regenerate plots cleanly.
- What did not finish yet: the full FID sweep is still the expensive part, so this deck stays conservative and treats the current numbers as preliminary.
- Comparison to the paper: our setup matches the paper's main practical question almost directly — how far can we cut the step count before quality degrades too much?
- Wall-clock time: the completed timing curve already makes the compute trade-off very obvious, especially when comparing 10–50 steps against 250–1000 steps.

The biggest thing we did not want to do here was overclaim. Until the SOL batch finishes, we keep the wording honest and preliminary.

Discussion of Results (Contd.)

- The submitted code is set up for practical reproducibility, not just a one-off run.
- benchmark.py supports a full run, resume mode, FID-only mode, and plot-only mode.
- setup_env.sbatch handles environment creation, and run_benchmark.sbatch is the main launch point on SOL.
- When the benchmark finishes, figures are generated automatically and written to figures/, which is exactly how the current draft plots were produced.

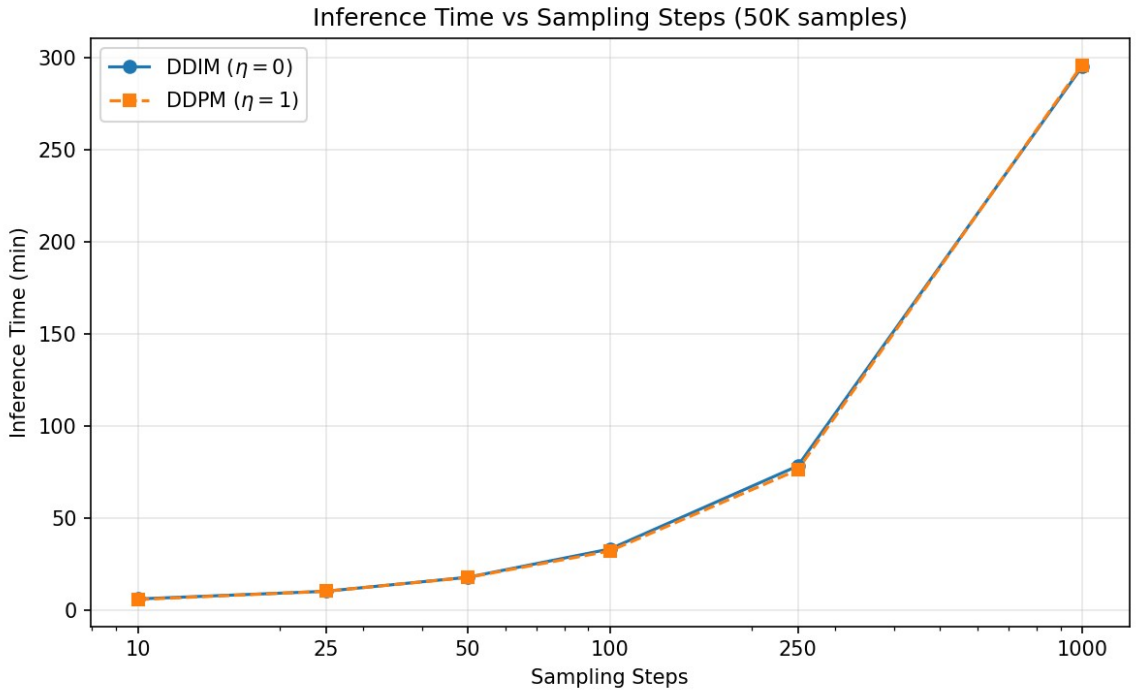
```
# submit and monitor
sbatch run_benchmark.sbatch
squeue -u $USER
tail -f benchmark_*.out

# pull results back
scp
<asurite>@solasu.edu:"~/ddim-benchmark/benchmark_results.
json" .
scp -r <asurite>@solasu.edu:"~/ddim-benchmark/figures" .
```

```
# rerun only what we need
python -u benchmark.py --resume
python -u benchmark.py --fid-only
python -u benchmark.py --plot-only
```

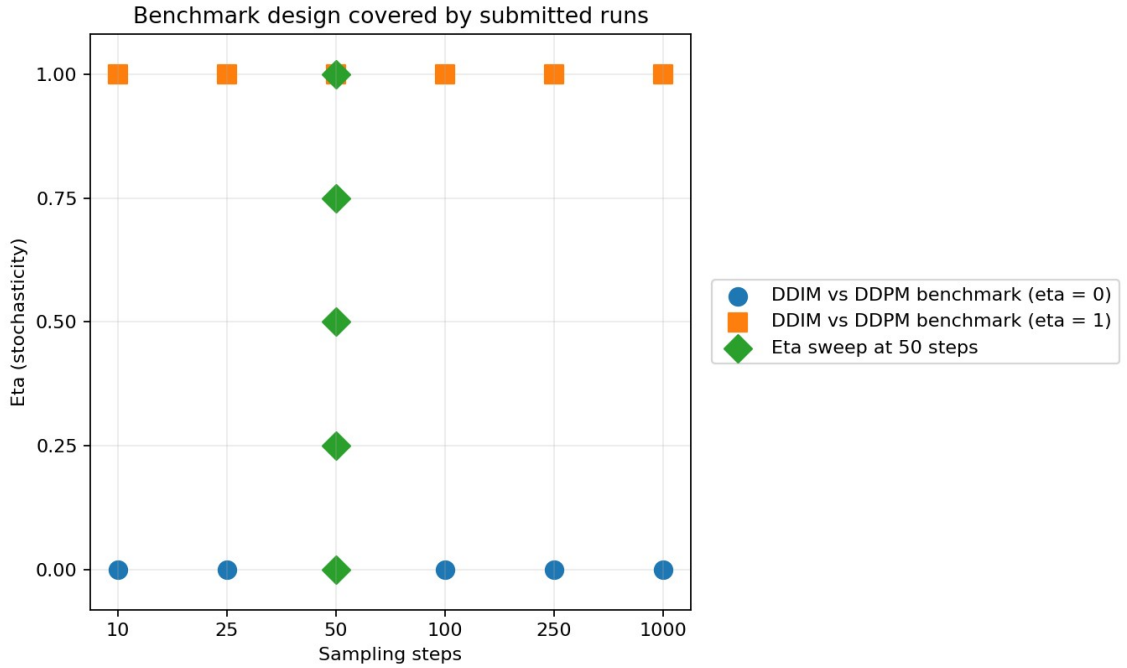
That script structure is part of the project result too: it makes the benchmark easy to continue while jobs are still running.

Visualization of Results

We only show finished visuals here. The second plot documents the exact submitted benchmark grid instead of inventing unfinished FID numbers.



Completed timing plot from the current run



Experiment map built from our submitted run settings

Pros and Cons of Approach

Why this approach should be adopted

- DDIM lets us reuse the same trained DDPM objective while making sampling much more flexible and potentially much faster at low step counts.
- The eta parameter gives a clean continuum between deterministic DDIM and stochastic DDPM-like sampling, which makes benchmarking straightforward.

Limitations / problems with this approach

- The speed/quality trade-off still has to be measured empirically; fewer steps can absolutely hurt sample fidelity.
- Full FID evaluation is expensive at 50K samples, so the cleanest quantitative story can take a while to finish on shared compute.

Course Concepts

- Variational learning / ELBO: the DDPM formulation starts from a latent-variable generative model and a variational training objective.
- Gaussian diffusion processes: forward noising, cumulative alpha-bar schedules, and the role of beta in controlling corruption over time.
- Deterministic vs stochastic sampling: DDIM shows that we can decouple the training loss from the exact reverse sampling path and study speed-quality trade-offs directly.

References

1. Song, J., Meng, C., and Ermon, S. 2021. Denoising Diffusion Implicit Models. In International Conference on Learning Representations (ICLR 2021). arXiv:2010.02502.

2. Ho, J., Jain, A., and Abbeel, P. 2020. Denoising Diffusion Probabilistic Models. In Advances in Neural Information Processing Systems (NeurIPS 2020). arXiv:2006.11239.

3. Heusel, M., Ramsauer, H., Unterthiner, T., Nessler, B., and Hochreiter, S. 2017. GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium. In Advances in Neural Information Processing Systems (NeurIPS 2017).

4. Krizhevsky, A. 2009. Learning Multiple Layers of Features from Tiny Images. Technical Report, University of Toronto.

5. Jennewein, D. M. et al. 2023. The Sol Supercomputer at Arizona State University. In Practice and Experience in Advanced Research Computing, 296-301. Association for Computing Machinery.

6. Ermon Group. Official DDIM implementation. GitHub repository.

7. EEE 598 team materials used for this deck: project proposal, DDIM math notes, submitted code/setup notes, and preliminary benchmark figures.